CSC209 Assignment 3
Multi-Process Monte Carlo π Estimator (Pipes)

MarkUs Group 54189
Wanru Cheng — 1010160335
Yuhan Kuang — 1010892759

April 6, 2026

1 Project Overview

This application is a distributed Monte Carlo simulator that estimates the value of π through parallel computation. It belongs to Category 1 — Multi-Process Application Using Pipes. The program uses a controller process that spawns a pool of worker processes to execute Monte Carlo simulations concurrently. Each worker independently performs random sampling trials to estimate π, and the controller aggregates results across workers to compute the final estimate.

Key features

- Parallel Monte Carlo simulation: workers perform independent random sampling using seeded pseudo-random number generation (`rand_r`) to simulate points in a unit square and count hits within a unit circle.
- Interactive command interface: the controller provides an interactive command-line interface for real-time feedback and control of simulations.
- Batched result streaming: simulation results are streamed in configurable batch chunks, allowing partial-result inspection and responsive feedback during long-running simulations.
- Graceful process management: handles worker lifecycle through signal handlers (`SIGCHLD`) and coordinated shutdown sequences.

Program runtime I/O

- `simulate [total_trials]:` Execute a Monte Carlo simulation with `total_trials` trials distributed across all workers. If `total_trials` is less than the number of workers, only the first `total_trials` workers will receive one trial (but unused workers are still active). For each `simulate` command, the controller prints aggregated hit counts across all workers and computes the estimated value of π using the formula $\pi \approx 4 \times$ (hits/total trials), as well as a 95% confidence interval and the error with respect to the actual π value.
- `setbatch [size]:` Set batch size (number of trials per partial result).

- `partial [on|off]`: Enable or disable tracing of partial results as they complete. Each partial result contains worker index, batch number, number of trials per batch, number of hits, and cumulative number of trials and hits.
- `stats`: Query and print aggregated worker statistics (worker PID and number of tasks completed).
- `workers`: Display the count of active/alive workers.
- `help`: Display available commands.
- `quit`: Gracefully shut down all workers and exit.

2 Build Instructions

- `make`: Build both the `controller` and `worker` executables.
- `make clean`: Remove compiled executables.

Command-line arguments: `./controller [num_workers [seed]]`

- `num_workers`: Number of worker processes to spawn (default: 4, maximum: 64).
- `seed`: Base random seed for worker RNG initialization (default: 42).

3 Architecture Diagram

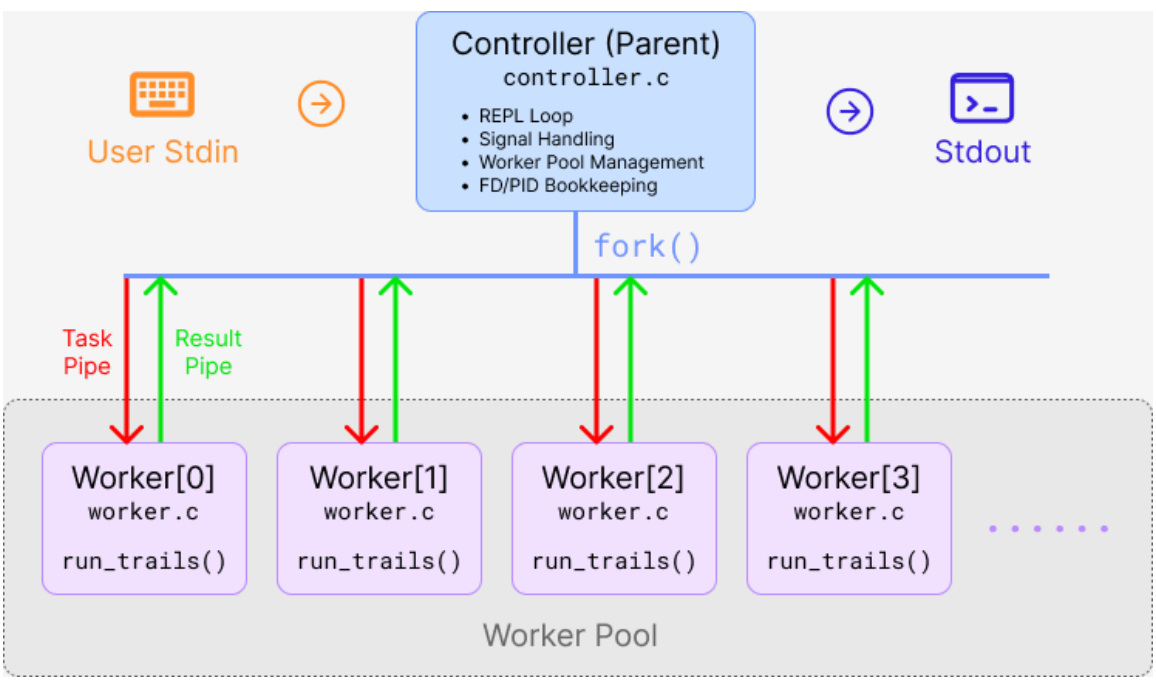

Figure 1: Architecture: controller, worker pool, and pipes.

Figure 1 shows the process and pipe structure for our Category 1 design: one controller and multiple workers, with dedicated communication channels between the parent and each child.

Key source files

File	Role
`montecarlo.h`	Shared protocol constants and `task_msg_t` / `result_msg_t`.
`controller.c`	`spawn_workers`, `run_simulation`, `query_worker_stats`, `shutdown_workers`, REPL (Read-Eval-Print Loop).
`worker.c`	Read loop, Monte Carlo trials, batched responses, status/error replies.
`Makefile`	`make` builds `controller` and `worker` with `-Wall -Wextra`.

4 Communication Protocol

This section specifies the binary protocol between the controller and workers. For each message type we state sender and receiver, encoding, semantics and invariants (including how many bytes are read, where the struct is stored in memory, and what the fields mean), and error handling. Unless noted otherwise, "controller" means `controller.c` and "worker" means `worker.c`.

4.1 Encoding common to every message

- Two struct types in `montecarlo.h`: `task_msg_t` (controller → worker on the task pipe) and `result_msg_t` (worker → controller on the result pipe).
- One logical message is one struct: `write_full(fd, &msg, sizeof msg)` to send; `read_full(fd, &msg, sizeof msg)` to receive into a local variable of that type (stack storage).
- Byte count is always `sizeof(task_msg_t)` or `sizeof(result_msg_t)`; there is no length prefix or delimiter.
- Fewer than `sizeof` bytes from `read_full` means failure (closed pipe or truncated read). Both sides retry on `EINTR`.
- Worker: EOF (End Of File) or short read on the task pipe ends the loop (no reply for a truncated message); failed `write_full` on the result pipe exits the process.

Listing 1: Struct layouts (`montecarlo.h L30-L58`)

```
typedef struct {
    uint32_t version;
    uint32_t task_id;
    uint32_t msg_type;
    uint32_t num_trials;
    uint32_t batch_trials;
} task_msg_t;

typedef struct {
    uint32_t version;
    uint32_t task_id;
    uint32_t msg_type;
    uint32_t num_trials;
    uint32_t num_hits;
    uint32_t batch_index;
    uint32_t batch_total;
```

```
    uint32_t tasks_completed;
    uint32_t error_code;
} result_msg_t;
```

4.2 Outgoing messages (`task_msg_t`)

`TASK_SIMULATE (montecarlo.h L10)`

- Sender / receiver: controller (`run_simulation`) → worker (`main` loop, task fd).
- Encoding: binary `task_msg_t`; `msg_type = TASK_SIMULATE`; `num_trials` = chunk; `batch_trials` from `setbatch` (the REPL (Read-Eval-Print Loop) keeps it positive; worker maps wire 0 to `SIM_BATCH_TRIALS` in `montecarlo.h`).
- Semantics / invariants: worker streams `RESULT_SIMULATE` partials whose per-message `num_trials` sum to the chunk; `batch_total` equals the controller's expected batch count, and `batch_index` runs strictly from 1 through `batch_total` (one partial per index).
- Error handling: failed controller `write_full` ⇒ `mark_worker_dead`. Invalid simulate request or version on the worker ⇒ one `RESULT_ERROR` (`EINVAL`/`EPROTO`), then the worker continues reading.

`TASK_STATUS_REQ (montecarlo.h L12)`

- Sender / receiver: controller (`query_worker_stats`) → worker.
- Encoding: binary `task_msg_t`; `msg_type = TASK_STATUS_REQ`; other numerics zero except `version`, `task_id`.
- Semantics / invariants: worker replies with one `RESULT_STATUS` (`task_id`, `tasks_completed`).
- Error handling: failed `write_full` when sending this request ⇒ `mark_worker_dead` (reply handling is under `RESULT_STATUS` / `RESULT_ERROR`).

`TASK_SHUTDOWN (montecarlo.h L11)`

- Sender / receiver: controller (`shutdown_workers`) → worker.
- Encoding: binary `task_msg_t`; `msg_type = TASK_SHUTDOWN`.
- Semantics / invariants: worker exits the read loop; no reply (equivalent to EOF (End Of File) on the task pipe).
- Error handling: `EPIPE` if the worker is already gone is fine; parent closes fds and `waitpid` each child (`ECHILD` if already reaped).

4.3 Incoming messages (`result_msg_t`)

`RESULT_SIMULATE (montecarlo.h L15)`

- Sender / receiver: worker (simulate branch) → controller (`run_simulation`).

- Encoding: binary `result_msg_t` per partial; `num_trials`, `num_hits`, `batch_index`, `batch_total`, `task_id`, `version`.
- Semantics / invariants: controller recomputes expected `batch_total` from the chunk and batch size, checks strictly increasing `batch_index`, and requires partial `num_trials` to sum to the chunk; it rejects `num_trials` = 0, `num_hits` > `num_trials`, or wrong `task_id`/`msg_type`/`version`.
- Error handling: short read or failed check ⇒ `mark_worker_dead`; other workers still count toward the same `simulate`.

`RESULT_STATUS (montecarlo.h L16)`

- Sender / receiver: worker (stats branch) → controller (`query_worker_stats`).
- Encoding: binary `result_msg_t`; `msg_type = RESULT_STATUS`; `tasks_completed` set.
- Semantics / invariants: `task_id` and `version` must match the outstanding stats request.
- Error handling: short read, wrong `task_id`, wrong `version`, or neither `RESULT_STATUS` nor `RESULT_ERROR` ⇒ `mark_worker_dead`. If `task_id` and `version` match and `msg_type` is `RESULT_ERROR`, the controller prints `error_code` and leaves the worker usable.

`RESULT_ERROR (montecarlo.h L17)`

- Sender / receiver: worker (bad task/version/type) → controller.
- Encoding: binary `result_msg_t`; `error_code` (`EINVAL`, `EPROTO`, ...).
- Semantics / invariants: during `run_simulation`, any `RESULT_ERROR` for the active `task_id` is a protocol violation (the controller expected only `RESULT_SIMULATE`).
- Error handling: `run_simulation` ⇒ `mark_worker_dead`. `query_worker_stats`: handled under `RESULT_STATUS` above when checks pass.

4.4 Summary table

Message	Direction	Primary code
`TASK_SIMULATE`	C → W	`run_simulation`
`TASK_STATUS_REQ`	C → W	`query_worker_stats`
`TASK_SHUTDOWN`	C → W	`shutdown_workers`
`RESULT_SIMULATE`	W → C	`run_simulation`
`RESULT_STATUS`	W → C	`query_worker_stats`
`RESULT_ERROR`	W → C	`worker.c` error paths

5 Concurrency Model

The implementation follows Category 1: a single parent process forks a fixed pool of worker children and uses pipes as the only inter-process channel. Coordination relies on blocking reads and writes on those pipes; the code does not use `select`, worker threads, or mutexes. The subsections below describe when the parent forks, when workers are treated as ready to accept work, and how child processes are collected and reaped.

5.1 Fork and `exec` (`spawn_workers`)

- The only `fork` calls are in `spawn_workers` (`controller.c L217-L237`), run once from `main` before the REPL starts.
- Per worker slot: create two pipes (task and result), then `fork`.
- Child: close every fd it should not hold (including other workers' pipe ends); `execl("./worker", ...)` with task read fd, result write fd, and seed `base_seed + i`.
- Parent: close the ends used by the child; record `task_write_fd`, `result_read_fd`, `pid`, and `alive` in `workers[i]`.
- No further `fork` or `exec` while processing user input.

5.2 Worker readiness after spawn

- No extra "ready" message is exchanged.
- When `spawn_workers` returns, each child is already running `worker` and blocked in `read_full` `(L43-L61)` on its task pipe (`worker.c main`).
- The controller treats a worker as ready to receive work once its fds are stored and `alive` is true; the first `simulate` or `stats` supplies bytes that unblock those reads.

5.3 Coordination and parallel execution

- Workers are separate processes and run in parallel; each waits on a blocking pipe read until the controller sends a message.
- The controller uses `write_full`(controller.c L333) to hand out work and `read_full` (controller.c L383) to collect replies; pipe blocking provides all synchronisation (no mutexes, no threads, no `select` in our design).
- For one `simulate`, the controller drains each worker's partial stream in turn (nested loops in `run_simulation`). This is correct because streams are independent and expected message counts are known; it avoids a `select`-based multiplexer.

5.4 Collecting and reaping child processes

- Normal exit: `shutdown_workers (L246-L280)` sends `TASK_SHUTDOWN (L70-L88)`, closes controller-side fds, then calls blocking `waitpid` for each stored child PID so every process is reaped.
- Unexpected exit: `sigchld_handler` uses `waitpid(..., WNOHANG)` to reap, clears `alive`, calls `close_worker_fds`, and zeros `pid` for that slot.
- Failed spawn: the `fail` path in `spawn_workers (L217-L237)` closes pipes, terminates and reaps any children already started, and resets `num_workers` before returning failure to `main`.

5.5 Worker count and idle workers

- `main` rejects `num_workers` outside $[3, 64]$.
- If the trial split gives a worker chunk size zero for one command, that worker gets no `TASK_SIMULATE` that round but stays alive and blocked for the next command; the pool remains multi-process.

6 Error Handling and Robustness

The following subsections cover five cases: three IPC/worker failures (pool creation, broken dispatch pipe, bad results), invalid REPL (Read-Eval-Print Loop) input, and long-run zombie plus fd cleanup (`SIGCHLD` and `quit`).

6.1 Pool creation failure (`pipe` or `fork`)

- Bad behaviour: the OS refuses a pipe or a fork while workers are being created.
- Handling: jump to `fail` in `spawn_workers`: close every pipe fd opened so far; `SIGTERM` and `waitpid` each child already forked; set `num_workers` to 0; return -1.
- Effect: `main` exits without leaving orphan workers or leaked descriptors.

6.2 Broken task pipe or dead worker during dispatch

- Bad behaviour: controller `write_full` to the task pipe fails (typically `EPIPE` if the child exited).
- Handling: call `mark_worker_dead (L59-L63)` to clear `alive` and close `task_write_fd` and `result_read_fd`. The process ignores `SIGPIPE (L585-L592)` so the error is returned from `write`, not by killing the parent.
- Effect: subsequent `simulate` and `stats` skip that index; remaining workers continue.

6.3 Truncated or protocol-violating results

- Bad behaviour: short `read_full`, or `result_msg_t` fields that violate the protocol (including batch metadata, trial totals, `num_hits` > `num_trials`, or zero `num_trials` in a partial).
- Handling: `mark_worker_dead` in `run_simulation`; stop reading that worker for this command.
- Effect: only validated partials affect the printed π estimate.

6.4 Invalid commands at the REPL (Read-Eval-Print Loop)

- Bad behaviour: unknown command name, invalid `simulate` argument (non-positive or out of range), or invalid `setbatch` value.
- Handling: the loop in `main` prints an error or usage line and continues without writing to worker pipes or forking.
- Effect: controller state stays consistent for the next line of input.

6.5 Zombies and file descriptor cleanup

- Bad behaviour: a worker dies unexpectedly while the controller keeps running.
- Handling: `SIGCHLD (L70-L88)` reaps and closes that worker's fds (§5); `shutdown_workers` closes any remaining fds and `waitpid` remaining children.
- Effect: no zombie buildup; clean exit on `quit`.

7 Team Contributions

- Wanru Cheng: Implemented code for Monte Carlo setbatch feature; recorded part 2 for demo video; wrote the second half of project documentation.
- Yuhan Kuang: Implemented code for Monte Carlo simulating feature; recorded part 1 for demo video; wrote the first half of project documentation.